JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ZoneId

java.lang.Object
    java.time.ZoneId

All Implemented Interfaces:

`Serializable`

Direct Known Subclasses:

`ZoneOffset`

```
public abstract sealed class ZoneId
extends Object
implements Serializable
permits ZoneOffset (not exhaustive)
```

A time-zone ID, such as `Europe/Paris`.

A `ZoneId` is used to identify the rules used to convert between an `Instant` and a `LocalDateTime`. There are two distinct types of ID:

- Fixed offsets - a fully resolved offset from UTC/Greenwich, that uses the same offset for all local date-times
- Geographical regions - an area where a specific set of rules for finding the offset from UTC/Greenwich apply

Most fixed offsets are represented by `ZoneOffset`. Calling `normalized()` on any `ZoneId` will ensure that a fixed offset ID will be represented as a `ZoneOffset`.

The actual rules, describing when and how the offset changes, are defined by `ZoneRules`. This class is simply an ID used to obtain the underlying rules. This approach is taken because rules are defined by governments and change frequently, whereas the ID is stable.

The distinction has other effects. Serializing the `ZoneId` will only send the ID, whereas serializing the rules sends the entire data set. Similarly, a comparison of two IDs only examines the ID, whereas a comparison of two rules examines the entire data set.

Time-zone IDs

The ID is unique within the system. There are three types of ID.

The simplest type of ID is that from `ZoneOffset`. This consists of 'Z' and IDs starting with '+' or '-'.

The next type of ID are offset-style IDs with some form of prefix, such as 'GMT+2' or 'UTC+01:00'. The recognised prefixes are 'UTC', 'GMT' and 'UT'.

The offset is the suffix and will be normalized during creation. These IDs can be normalized to a `ZoneOffset` using `normalized()`.

The third type of ID are region-based IDs. A region-based ID must be of two or more characters, and not start with 'UTC', 'GMT', 'UT' '+' or '-'. Region-based IDs are defined by configuration, see `ZoneRulesProvider`. The configuration focuses on providing the lookup from the ID to the underlying `ZoneRules`.

Time-zone rules are defined by governments and change frequently. There are a number of organizations, known here as groups, that monitor time-zone changes and collate them. The default group is the IANA Time Zone Database (TZDB). Other organizations include IATA (the airline industry body) and Microsoft.

Each group defines its own format for the region ID it provides. The TZDB group defines IDs such as 'Europe/London' or 'America/New_York'. TZDB IDs take precedence over other groups.

It is strongly recommended that the group name is included in all IDs supplied by groups other than TZDB to avoid conflicts. For example, IATA airline time-zone region IDs are typically the same as the three letter airport code. However, the airport of Utrecht has the code 'UTC', which is obviously a conflict. The recommended format for region IDs from groups other than TZDB is 'group~region'. Thus if IATA data were defined, Utrecht airport would be 'IATA~UTC'.

Serialization

This class can be serialized and stores the string zone ID in the external form. The `ZoneOffset` subclass uses a dedicated format that only stores the offset from UTC/Greenwich.

A `ZoneId` can be deserialized in a Java Runtime where the ID is unknown. For example, if a server-side Java Runtime has been updated with a new zone ID, but the client-side Java Runtime has not been updated. In this case, the `ZoneId` object will exist, and can be queried using `getId`, `equals`, `hashCode`, `toString`, `getDisplayName` and `normalized`. However, any call to `getRules` will fail with `ZoneRulesException`. This approach is designed to allow a `ZonedDateTime` to be loaded and queried, but not modified, on a Java Runtime with incomplete time-zone information.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The `equals` method should be used for comparisons.

Implementation Requirements:

This abstract sealed class permits two implementations, both of which are immutable and thread-safe. One implementation models region-based IDs, the other is `ZoneOffset` modelling offset-based IDs. This difference is visible in serialization.

Sealed Class Hierarchy Graph:

Sealed class hierarchy graph for ZoneId

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
`static final Map<String,String>`	**`SHORT_IDS`**	A map of zone overrides to enable the short time-zone names to be used.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`boolean`	**`equals`**`(Object obj)`	Checks if this time-zone ID is equal to another time-zone ID.
`static ZoneId`	**`from`**`(TemporalAccessor tempor`	Obtains an instance of `ZoneId` from a temporal object.
`static Set<String>`	**`getAvailableZoneIds`**`()`	Gets the set of available zone IDs.
`String`	**`getDisplayName`**`(TextStyle style, Locale locale)`	Gets the textual representation of the zone, such as 'British Time' or '+02:00'.
`abstract String`	**`getId`**`()`	Gets the unique time-zone ID.

abstract ZoneRules	**getRules**()	Gets the time-zone rules for this ID allowing calculations to be performed.
int	**hashCode**()	A hash code for this time-zone ID.
ZoneId	**normalized**()	Normalizes the time-zone ID, returning a `ZoneOffset` where possible.
static ZoneId	**of**(String zoneId)	Obtains an instance of `ZoneId` from an ID ensuring that the ID is valid and available for use.
static ZoneId	**of**(String zoneId, Map<String, String> aliasMap)	Obtains an instance of `ZoneId` using its ID using a map of aliases to supplement the standard zone IDs.
static ZoneId	**ofOffset**(String prefix, ZoneOffset offset)	Obtains an instance of `ZoneId` wrapping an offset.
static ZoneId	**systemDefault**()	Gets the system default time-zone.
String	**toString**()	Outputs this zone as a `String`, using the ID.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

SHORT_IDS

```
public static final Map<String,String> SHORT_IDS
```

A map of zone overrides to enable the short time-zone names to be used.

Use of short zone IDs has been deprecated in `java.util.TimeZone`. This map allows the IDs to continue to be used via the `of(String, Map)` factory method.

This map contains a mapping of the IDs that is in line with TZDB 2024b and later, where 'EST', 'MST' and 'HST' map to IDs which do not include daylight savings since 1970. This mapping may change in update releases in support of new versions of TZDB.

This maps as follows:

- ACT - Australia/Darwin
- AET - Australia/Sydney
- AGT - America/Argentina/Buenos_Aires
- ART - Africa/Cairo
- AST - America/Anchorage
- BET - America/Sao_Paulo
- BST - Asia/Dhaka
- CAT - Africa/Harare
- CNT - America/St_Johns
- CST - America/Chicago
- CTT - Asia/Shanghai
- EAT - Africa/Addis_Ababa
- ECT - Europe/Paris
- EST - America/Panama
- HST - Pacific/Honolulu
- IET - America/Indiana/Indianapolis
- IST - Asia/Kolkata
- JST - Asia/Tokyo
- MIT - Pacific/Apia
- MST - America/Phoenix
- NET - Asia/Yerevan
- NST - Pacific/Auckland
- PLT - Asia/Karachi
- PNT - America/Phoenix
- PRT - America/Puerto_Rico
- PST - America/Los_Angeles
- SST - Pacific/Guadalcanal
- VST - Asia/Ho_Chi_Minh

The map is unmodifiable.

Method Details

systemDefault

```
public static ZoneId systemDefault()
```

Gets the system default time-zone.

This queries `TimeZone.getDefault()` to find the default time-zone and converts it to a `ZoneId`. If the system default time-zone is changed, then the result of this method will also change.

Returns:

the zone ID, not null

Throws:

`DateTimeException` - if the converted zone ID has an invalid format

`ZoneRulesException` - if the converted zone region ID cannot be found

getAvailableZoneIds

```
public static Set<String> getAvailableZoneIds()
```

Gets the set of available zone IDs.

This set includes the string form of all available region-based IDs. Offset-based zone IDs are not included in the returned set. The ID can be passed to `of(String)` to create a `ZoneId`.

The set of zone IDs can increase over time, although in a typical application the set of IDs is fixed. Each call to this method is thread-safe.

Returns:

a modifiable copy of the set of zone IDs, not null

of

```
public static ZoneId of(String zoneId,
                        Map<String,String> aliasMap)
```

Obtains an instance of `ZoneId` using its ID using a map of aliases to supplement the standard zone IDs.

Many users of time-zones use short abbreviations, such as PST for 'Pacific Standard Time' and PDT for 'Pacific Daylight Time'. These abbreviations are not unique, and so cannot be used as IDs. This method allows a map of string to time-zone to be setup and reused within an application.

Parameters:

`zoneId` - the time-zone ID, not null

`aliasMap` - a map of alias zone IDs (typically abbreviations) to real zone IDs, not null

Returns:

the zone ID, not null

Throws:

`DateTimeException` - if the zone ID has an invalid format

`ZoneRulesException` - if the zone ID is a region ID that cannot be found

of

```
public static ZoneId of(String zoneId)
```

Obtains an instance of `ZoneId` from an ID ensuring that the ID is valid and available for use.

This method parses the ID producing a `ZoneId` or `ZoneOffset`. A `ZoneOffset` is returned if the ID is 'Z', or starts with '+' or '-'. The result will always be a valid ID for which `ZoneRules` can be obtained.

Parsing matches the zone ID step by step as follows.

- If the zone ID equals 'Z', the result is `ZoneOffset.UTC`.
- If the zone ID consists of a single letter, the zone ID is invalid and `DateTimeException` is thrown.
- If the zone ID starts with '+' or '-', the ID is parsed as a `ZoneOffset` using `ZoneOffset.of(String)`.
- If the zone ID equals 'GMT', 'UTC' or 'UT' then the result is a `ZoneId` with the same ID and rules equivalent to `ZoneOffset.UTC`.
- If the zone ID starts with 'UTC+', 'UTC-', 'GMT+', 'GMT-', 'UT+' or 'UT-' then the ID is a prefixed offset-based ID. The ID is split in two, with a two or three letter prefix and a suffix starting with the sign. The suffix is parsed as a `ZoneOffset`. The result will be a `ZoneId` with the specified UTC/GMT/UT prefix and the normalized offset ID as per `ZoneOffset.getId()`. The rules of the returned `ZoneId` will be equivalent to the parsed `ZoneOffset`.
- All other IDs are parsed as region-based zone IDs. Region IDs must match the regular expression `[A-Za-z][A-Za-z0-9~/._+-]+` otherwise a `DateTimeException` is thrown. If the zone ID is not in the configured set of IDs, `ZoneRulesException` is thrown. The detailed format of the region ID depends on the group supplying the data. The default set of data is supplied by the IANA Time Zone Database (TZDB). This has region IDs of the form '{area}/{city}', such as 'Europe/Paris' or 'America/New_York'. This is compatible with most IDs from `TimeZone`.

Parameters:

`zoneId` - the time-zone ID, not null

Returns:

the zone ID, not null

Throws:

`DateTimeException` - if the zone ID has an invalid format

`ZoneRulesException` - if the zone ID is a region ID that cannot be found

ofOffset

```
public static ZoneId ofOffset(String prefix,
                              ZoneOffset offset)
```

Obtains an instance of `ZoneId` wrapping an offset.

If the prefix is "GMT", "UTC", or "UT" a `ZoneId` with the prefix and the non-zero offset is returned. If the prefix is empty `""` the `ZoneOffset` is returned.

Parameters:

`prefix` - the time-zone ID, not null

`offset` - the offset, not null

Returns:

the zone ID, not null

Throws:

`IllegalArgumentException` - if the prefix is not one of "GMT", "UTC", or "UT", or ""

from

```
public static ZoneId from(TemporalAccessor temporal)
```

Obtains an instance of `ZoneId` from a temporal object.

This obtains a zone based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ZoneId`.

A `TemporalAccessor` represents some form of date and time information. This factory converts the arbitrary temporal object to an instance of `ZoneId`.

The conversion will try to obtain the zone in a way that favours region-based zones over offset-based zones using `TemporalQueries.zone()`.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `ZoneId::from`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the zone ID, not null

Throws:

`DateTimeException` - if unable to convert to a `ZoneId`

getId

```
public abstract String getId()
```

Gets the unique time-zone ID.

This ID uniquely defines this object. The format of an offset based ID is defined by `ZoneOffset.getId()`.

Returns:

the time-zone unique ID, not null

getDisplayName

```
public String getDisplayName(TextStyle style,
                             Locale locale)
```

Gets
the textual representation of the zone, such as 'British Time' or '+02:00'.

This returns the textual name used to identify the time-zone ID, suitable for presentation to the user. The parameters control the style of the returned text and the locale.

If no textual mapping is found then the `full ID` is returned.

Parameters:

`style` - the length of the text required, not null

`locale` - the locale to use, not null

Returns:

the text value of the zone, not null

getRules

```
public abstract ZoneRules getRules()
```

Gets the time-zone rules for this ID allowing calculations to be performed.

The rules provide the functionality associated with a time-zone, such as finding the offset for a given instant or local date-time.

A time-zone can be invalid if it is deserialized in a Java Runtime which does not have the same rules loaded as the Java Runtime that stored it. In this case, calling this method will throw a `ZoneRulesException`.

The rules are supplied by `ZoneRulesProvider`. An advanced provider may support dynamic updates to the rules without restarting the Java Runtime. If so, then the result of this method may change over time. Each individual call will be still remain thread-safe.

`ZoneOffset` will always return a set of rules where the offset never changes.

Returns:

the rules, not null

Throws:

`ZoneRulesException` - if no rules are available for this ID

normalized

```
public ZoneId normalized()
```

Normalizes the time-zone ID, returning a `ZoneOffset` where possible.

The returns a normalized `ZoneId` that can be used in place of this ID. The result will have `ZoneRules` equivalent to those returned by this object, however the ID returned by `getId()` may be different.

The normalization checks if the rules of this `ZoneId` have a fixed offset. If they do, then the `ZoneOffset` equal to that offset is returned. Otherwise `this` is returned.

Returns:

the time-zone unique ID, not null

equals

```
public boolean equals(Object obj)
```

Checks if this time-zone ID is equal to another time-zone ID.

The comparison is based on the ID.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other time-zone ID

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this time-zone ID.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Outputs this zone as a `String`, using the ID.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this time-zone ID, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.